

YAD!

YET ANOTHER DRUMMER!



YAD! (Yet Another Drummer)

YAD! v.1.0.0

User Manual

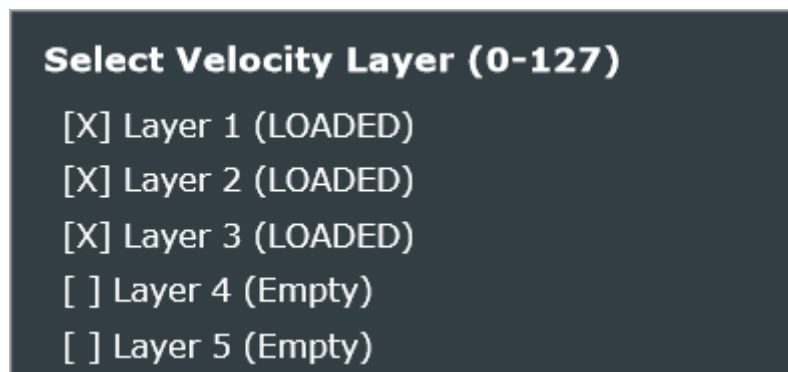


2. The Interface

- **The Drum Kit View:** The top section features an interactive visual representation of a drum kit. You can trigger sounds by clicking the pads or using your MIDI keyboard (mapped to standard GM Drum MIDI notes).
- **The Mixer (Bottom Panel):** * **OUT [1-8]:** Direct output routing selector.
 - **LOAD:** Sample and layer management.
 - **Gain Sliders:** Horizontal faders to balance the volume of each drum element.

3. Creating Your Library (Multi-Layer System)

Unlike basic samplers, Drum Simulator supports up to **5 velocity layers** per drum pad. This allows for a "human" feel where different samples are triggered based on how hard you hit the key.



How to Load Samples:

1. Click the **LOAD** button on a specific channel (e.g., Kick).
2. A menu will appear asking you to choose a **Velocity Layer (1-5)**.
3. Select a file (.wav or .aiff).
4. **Automatic Mapping:** The engine automatically redistributes the MIDI velocity range (0-127).
 - If you load **1 sample**, it plays at all velocities.
 - If you load **3 samples**, the engine splits the range into roughly 0-42, 43-85, and 86-127.
5. The layer already occupied by a loaded sample is indicated with a check mark on the left. If you load even just one sample on a single layer, the red dot on the pad turns green.

Pro Tip for Realism:

To create a professional-sounding kit, load a "Soft" hit in Layer 1, a "Medium" hit in Layer 3, and a "Hard/Power" hit in Layer 5. The engine will handle the cross-fading and selection logic seamlessly.

4. Managing Kits (Save & Load)

- **SAVE KIT:** Click this in the top right to export your entire setup as a .xzkit file. This file remembers the paths to all 40 possible sample slots (8 pads x 5 layers) and your gain settings.
- **LOAD KIT:** Use this to recall a previously saved .yadkit.
 - *Note: Ensure your audio samples remain in the same folder location on your hard drive, as the kit file stores file paths.*

5. Audio Routing

Each drum element can be routed to a specific output in your DAW. Click the **OUT** button above the load button to toggle between:

- **Main:** Stereo Master Mix.
- **Direct:** Individual mono/stereo outputs for separate processing (EQ, Compression) in your DAW mixer.

SAMPLES REQUIREMENTS

1. Supported Formats

Your current code, via `JUCE::AudioFormatManager`, supports exactly what we recorded in the constructor:

- Supported: .wav, .aif / .aiff, .flac.
- Not supported (currently): .ape and WavPack. JUCE doesn't include these formats natively; external libraries would be needed to support them. I'd say sticking with WAV, AIFF, and FLAC is the standard professional choice.

2. Sample Rate and Bit Depth

The engine is extremely flexible because JUCE **resamples on the fly** if necessary:

- Sample Rate: Supports any rate (**44.1, 48, 88.2, 96, 192 kHz**). If the sample rate is 96kHz and your sound card is 44.1kHz, the plugin will still read the file correctly.
- Bit Depth: Supports **16-bit, 24-bit, and 32-bit (both integer and floating point)**. Internally, the plugin converts everything to 32-bit float for audio processing, ensuring the highest possible fidelity.

3. File Naming

No limitations. The system isn't based on the file name (like some samplers that look for "velocity" in the name), but on the location where you choose to load it via the interface. You can name the files whatever you like.

4. Hybrid Instruments and Experimentation

Absolutely! The plugin doesn't "know" that the sound is a snare drum.

- You can load a bass drum on Layer 1 and a human scream on Layer 5: at low velocity, you'll hear the bass; if you hit hard, you'll hear the scream.
- You can create hybrid kits (e.g., real kick drum, electronic snare drum, reverse-recorded cymbals). It's the perfect tool for creative sound design.

5. File Length

Technically, there's no limit (you could load a 10-minute file), but remember that:

- The plugin loads the entire sample into RAM.
- If you load 40 very long samples (8 pads x 5 layers), memory usage will increase dramatically. For a drum kit (single hits), this is never a problem.

6. Loop Support

Here we need to make an important technical distinction:

- Loading: You can load a file containing a loop.
- Playback: The plugin is currently a One-Shot Sampler. When it receives a MIDI note, it plays the file from start to finish (or until the sample ends).
- Current Limitations: We haven't implemented the "Loop Point" (i.e., the part that repeats as long as you hold down the key). If you load a 2-bar drum loop, it will only play it all once. No slicing system.
- Multi-layer: Yes, you can load different loops on different layers of the same pad (e.g., a calm shaker loop at low velocity and a frenetic one at high velocity).

CURRENT MIDI MAPPING OF THE PADS

Pad	MIDI Note	Note Number	Keyboard key (Standard)
KICK	C1	36	Do 1
SNARE	D1	38	Re 1
HI-HAT	F#1	42	Fa# 1
CRASH	C#2	49	Do# 2
TOM 1	A1	45	La 1
TOM 2	B1	47	Si 1
TOM 3	C2	48	Do 2
RIDE	D#2	51	Re# 2

PAD MAP AND EXTRA MAP

1. The Multi-Layer Sampling System

Each of the 8 Standard Pads (Kick, Snare, etc.) now supports up to **5 velocity layers**.

- **How to load:** Click the "**L**" (**Load**) button on any pad. A menu will appear showing 5 slots.
- **Visual Feedback:**
 - [] Layer X (Empty): No sample loaded.
 - [X] Layer X (LOADED): A sample is active.
- **Velocity Logic:** The engine automatically splits the MIDI velocity (0-127) across the loaded layers. If you load only one, it covers the full range. If you load five, each handles a specific dynamic range for maximum realism.

2. The "Extra Map" Configuration

The Extra Map allows you to expand your kit beyond the 8 standard pads, utilizing the rest of the MIDI keyboard (60 additional notes across 5 octaves).

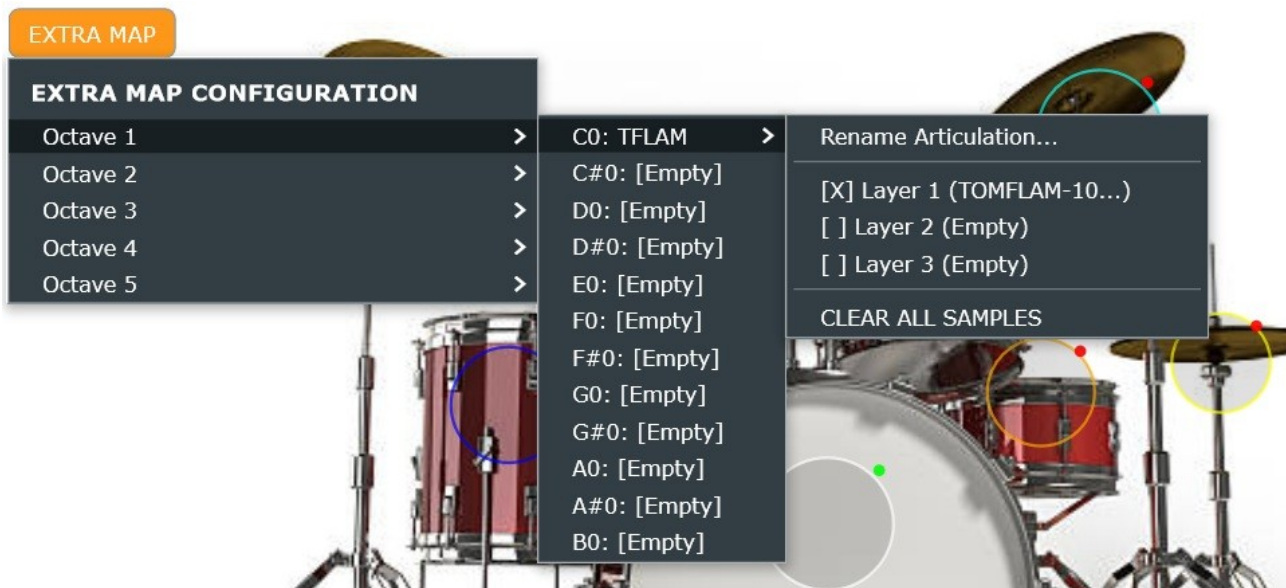
Mapping a New Articulation:

1. Click the **EXTRA MAP** button.
2. Navigate through the **Octave sub-menus** (Octave 1 to 5).
3. **Reserved Notes:** You will notice some notes are marked as [RESERVED FOR . . .]. These are the notes mapped to your main 8 Pads (e.g., C1 for Kick). These cannot be overwritten to prevent trigger conflicts.
4. **Empty Notes:** Click any [Empty] note to open the **Articulation Dialog**. Enter a short name (max 5 characters, e.g., "FLAM" or "COWBL").

Loading Audio to Extra Notes:

Once a note is named, clicking it again in the menu reveals a specialized sub-menu:

- **Rename:** Change the label.
- **Layer 1-3:** Extra articulations support **3 velocity layers** (Low, Mid, High).
- **Clear All Samples:** Resets the audio buffers for that specific note.



3. Presets and File Structure (.yadkit)

The new 1.0.0 version introduces the `.yadkit` format, an XML-based preset file that stores your entire configuration.

- **Saving:** Click **SAVE KIT**. This saves the paths to your audio files, your gain settings, output routing, and all Extra Map data.
- **Loading:** Click **LOAD KIT**. YAD will attempt to rebuild the entire kit.
- **Best Practice:** Keep your drum samples in organized folders. Since `.yadkit` stores *absolute paths*, moving your sample library to a different drive might require re-loading the samples.

DAW Test Checks:

When loading the plugin into a DAW (Reaper, Ableton, etc.):

1. Multi-Output: Check if the DAW sees the 8 separate outputs. This usually requires "activating" them in the plugin settings within the DAW.
2. Latency: It should be zero (or better yet, equal to the latency setting on your sound card).

Technical Implementation Details

For developers or advanced users interested in the "under the hood" solutions:

The Engine: DrumVoice & DrumLayer

The core of the plugin uses a custom-built playback engine. Each drum pad is a DrumVoice containing a dynamic `std::vector` of DrumLayer objects. Each layer holds its own AudioBuffer, allowing for zero-latency switching between velocity zones.

Dynamic Velocity Redistribution

The code employs an automated linear distribution algorithm. When a sample is added or removed, the engine recalculates the `lowVel` and `highVel` integers for every active layer:

$$\text{VelocityRange} = \frac{128}{\text{LoadedLayers}}$$

This ensures the player never encounters "dead zones" in their MIDI performance.

State Management via JSON

The `.xzkit` format is based on **JSON (JavaScript Object Notation)**. We utilized `juce::DynamicObject` and `juce::JSON` to create a human-readable save state. This makes the library files lightweight and easy to troubleshoot or manually edit if a file path changes.

Geometry Logic (SUI)

The GUI uses a **Subtractive Bounds** approach. By utilizing `removeFromBottom` and `removeFromLeft`, the interface ensures that the Mixer occupies a fixed 120-pixel safety zone, while the drum kit image scales proportionally to fill the remaining top area, preventing UI "crunching" in small DAW windows.

Technical Insights (Under the Hood)

For those interested in the architecture of YAD, several advanced C++ and JUCE solutions were implemented:

- **Asynchronous UI Pattern:** To prevent DAW freezes, all file browsing and dialog windows (like the naming of extra articulations) use **Modal-less Async Callbacks**. This ensures the audio thread and the GUI thread never block each other.
- **Dynamic Mapping via `std::map`:** Unlike the standard pads which use fixed arrays, the Extra Map utilizes a `[ std::map<int, std::unique_ptr<ExtraArticulation>> ]`. This allows for a memory-efficient "sparse" structure: memory is only allocated for the MIDI notes you actually use.
- **Smart Menu ID Routing:** The popup menu uses a mathematical ID offset system (e.g., IDs 2000+ for layers, 5000+ for clearing). This allows a single callback function to handle hundreds of potential menu items dynamically without complex switch-case structures.

- **XML Serialization:** The `.yadkit` format uses a hierarchical XML structure. This was chosen over binary formats to ensure that presets remain human-readable for debugging and future-proof as the plugin evolves.
-
-
-